TripNest AI: Deployment Guide

Version 1.0 · 2 November 2026.

> **Status: NOT DEPLOYED.** The Docker and Render files are prepared and the production-mode start was verified locally, but nothing has been deployed because it needs your hosting account. There is **no public URL** yet. Do not present one until the live checks in section 5 pass.

1. What runs in production

One Docker container serves the API **and** the web app (same origin, so no CORS setup), backed by PostgreSQL. On start it runs `alembic upgrade head`, seeds the catalogue if the database is empty, loads the saved crowd model from `backend/models/` and serves on `$PORT`. Health check: `/api/health`.

2. Environment variables

Set these in the hosting dashboard. **Never commit real values.**

Variable	Required	Value
`ENV`	yes	`production` (the app refuses to start without `SECRET_KEY` and `DATABASE_URL`)
`SECRET_KEY`	yes	32 or more random characters. Render can generate it (`render.yaml`)
`DATABASE_URL`	yes	PostgreSQL connection string. `postgres://` and `postgresql://` forms are accepted
`WIKIMEDIA_CONTACT`	recommended	your GitHub URL or email. Wikimedia blocks clients that do not identify themselves, and place photos then fail
`PHOTO_PROVIDER`	no	`wikimedia` (default) or `off`
`APP_TZ`	no	`Asia/Kolkata`
`ALLOWED_ORIGINS`	no	leave empty (same origin)

No API key is needed for maps (OpenStreetMap tiles) or photos (Wikimedia).

3. Deploy on Render (recommended, because `render.yaml` exists)

1. Push the repository to GitHub (the `.env` file is git-ignored; check `git status` shows no secrets).
2. On render.com, create an account, choose **New, Blueprint**, and select the repository. Render reads `render.yaml` and creates a Docker web service and a PostgreSQL database.
3. When asked, enter `WIKIMEDIA_CONTACT`.
4. Wait for the build and for the health check to pass.
5. Open the service URL.

Costs and limits (check on render.com before relying on them; plans change): free web services sleep when idle, so the first request after a pause can take about a minute, so open the URL a few minutes before a demo; free databases have had storage limits and an expiry period. If the Blueprint rejects a plan name, choose the cheapest current plan in the dashboard. Hosting costs, if any, are yours to approve.

4. Other hosts

Any host that runs a Docker image works: build with `docker build -t tripnest-ai .`, set the variables above and expose `$PORT`. Docker is not installed on the development machine, so **the image build itself has not been tested**; the application was tested by running it directly with the production settings (section 6).

5. Verification checklist (do this on the live URL before announcing it)

Check	Pass when
`GET /api/health`	`status: ok`, `database: ok`, `model.loaded: true`
Home page	loads with the hero photograph; city suggestions appear
Journey 1	Type Chennai, pick a date, Explore Destinations: popular places with photos and crowd chips, best-time panel, hidden gems, map with markers
Journey 2	View details shows a prediction, chart, best time and 7-day tiles
Journey 3	Map filters (Hidden gems, Low) change the markers; a popup opens with View Details
Journey 4	Deny browser location, type a city, results appear
Phone	open the URL on a phone; nothing scrolls sideways
`/docs`	the API documentation opens
Response	recommendations return in a few seconds even on first use

Record the real URL and the date of this check in `TEST_RESULTS.md` and `IMPLEMENTATION_STATUS.md`. The browser tests can be pointed at a live site with `E2E_BASE=https://your-url python -m pytest e2e` (they were written for a local server; run them only against a deployment you own).

6. What was verified locally for production

Executed on 2 November 2026 with `ENV=production`: startup is refused when `SECRET_KEY` is empty, when it is shorter than 32 characters, and when `DATABASE_URL` is missing; `alembic upgrade head` creates the schema on an empty database; the server then starts, `/api/health` reports ok, HSTS and other security headers are sent, a recommendation request succeeds and `/docs` opens. **Not verified:** the Docker build, PostgreSQL (SQLite was used), and any hosting provider.

7. Updating, retraining and rollback

Redeploy by pushing to the repository. To retrain, run `python -m scripts.train_model` locally and commit the new `backend/models/` files (training needs about 1.3 GB of memory, which is why the server loads the saved model instead of training at start). To roll back, redeploy the previous commit; the schema has one migration.

8. Known limitations

Demonstration crowd data; photos and map tiles need outbound internet; single-instance in-memory forecast cache; no monitoring or alerting; no HTTPS configuration is needed on Render (it terminates TLS) but other hosts must provide it.

Photo columns and image data (added in the redesign)

The migration `8a2f26c2f8b6_destination_photos` adds the `image_*` columns; `alembic upgrade head` applies it. Photographs are stored as Wikimedia URLs with credit and licence, loaded from `backend/app/data/images.json` when the catalogue is seeded (no files are stored). To refresh them, run `python -m scripts.fetch_images` with `WIKIMEDIA_CONTACT` set to your contact address, which Wikimedia requires. If a photo host is unreachable the page shows a neutral placeholder.